Vulkax Atlas: Route-Predictive Joint Resource Allocation for a Vulkan Globe and Navigation Runtime

Swayam Singal

July 2026

Abstract

Planet-scale navigation renderers must place useful content in memory before the camera reaches it, while simultaneously controlling frame time, upload bandwidth, geometric detail, and lighting error. Visibility- or distance-only streaming reacts to the current view but ignores a route's strong prediction of future demand. This paper presents Vulkax Atlas, a C++20 Vulkan globe and navigation research runtime built on BEACON and GeoBEACON. Atlas combines double-precision WGS84 and Earth-centered Earth-fixed coordinates, six-face cube-quadtree selection, cancellable file/HTTP/SQLite streaming, offline regional packs, normalized search/routing/transit/traffic providers, and a route-predictive budget controller. The controller allocates tile residency, upload bandwidth, geometric utility, and bounded diffuse-light work using route probability, time-to-arrival, semantics, visibility, and measured frame pressure. A deterministic constrained scheduler experiment produces raw frame rows and seven ablations. In this synthetic stress fixture, distance-only and geometry-only policies each recorded 644 route-corridor deadline misses, velocity-only recorded 204, route-only recorded 8, and route-plus-semantics and full Atlas each recorded 4. These values are explicitly classified as analytical scheduler simulation rather than Vulkan performance measurements. The artifact preserves the checked GeoBEACON renderer, exact diffuse reference captures, Vulkan query-pool timing, five reproducible region manifests, and per-source attribution.

1 Introduction

A globe renderer has a future-information problem. The current camera identifies visible content, but a turn-by-turn route identifies content likely to become visible seconds later. A naive streamer either requests too late, creating low-detail transitions and missing labels, or requests too broadly, wasting bandwidth and memory. The same route also changes the value of rendered content: an upcoming intersection, maneuver label, station, or landmark matters more than an equally sized object outside the route corridor.

Vulkax Atlas tests the following hypothesis:

> *Joint route- and semantic-aware allocation of geometry detail, streaming bandwidth, memory, and lighting accuracy reduces route-corridor deadline misses or wasted prefetch bytes compared with distance-only and velocity-only control under an equal budget.*

The contribution is the unified allocation policy and its ablations. Globe rendering, cube maps, screen-space error (SSE), clustered lighting, route engines, and HTTP caching are supporting systems rather than novelty claims.

Atlas extends GeoBEACON from one checked neighborhood to a planet-ready coordinate and content contract. Existing BEACON technique identifiers and GeoBEACON datasets remain regression baselines. The renderer distinguishes three evidence classes: Vulkan GPU timestamps, Vulkan CPU fallback, and analytical scheduler simulation. Conservative omitted-light bounds also remain separate from rendered MSE, PSNR, SSIM, and maximum pixel error.

2 Prior Work and Positioning

Hierarchical terrain and globe systems routinely use quadtree refinement and SSE. OGC 3D Tiles 1.1 standardizes hierarchical spatial content, implicit tiling, glTF payloads, feature identifiers, and structural metadata [1]. OpenStreetMap (OSM) provides global vector geometry and replication diffs under the ODbL [2, 3]. Pelias, Valhalla, and OpenTripPlanner provide open geocoding, road routing, and multimodal transit foundations [4, 5, 6].

Game engines also stream world sectors using visibility, distance, predicted movement, authored importance, and mission state. Atlas does not claim novelty for predictive streaming in isolation. Its research claim is narrower: one transparent controller combines route probability, time-to-arrival, maneuver and semantic importance, geometry and upload cost, and BEACON's measured lighting/error controls; each signal is independently disabled in a reproducible ablation.

Clustered and tiled shading established spatial light lists. Lightcuts-style systems established bounded many-light approximation, while grid-based reservoirs such as ReGIR target much larger stochastic many-light workloads [7]. Atlas retains BEACON's deterministic finite-radius point-light scope. Formal error accounting covers opaque, unshadowed diffuse direct light; visual specular and shadows are outside the guarantee.

3 Planetary Coordinate Model

Persistent positions use WGS84 latitude ϕ, longitude λ, and ellipsoidal height h. With semi-major axis a, eccentricity e, and prime-vertical radius

$$N(\phi) = \frac{a}{\sqrt{1 - e^2 \sin^2 \phi}},$$

the Earth-centered Earth-fixed (ECEF) position is

$$x = (N + h)\cos\phi\cos\lambda, \tag{1}$$

$$y = (N + h)\cos\phi\sin\lambda, \tag{2}$$

$$z = (N(1 - e^2) + h)\sin\phi. \tag{3}$$

Atlas retains these values in double precision. Regional tiles use an East-North-Up frame centered at a geodetic origin. Render inputs subtract the camera in double precision before conversion to float, avoiding centimeter-scale motion on top of million-meter coordinates.

The globe uses six cube faces. A normalized direction is projected to the dominant axis, producing face-local $(u, v) \in [-1, 1]^2$. A tile key stores face, level, integer x, y, and semantic layer. Parent and four-child operations are exact integer transforms. Property and unit tests cover all faces, geodetic round trips, local frames, poles, and invalid keys.

4 Globe Selection

Each frame begins with six root nodes. A tile's center direction maps to the WGS84 surface; corner directions bound its angular radius. Atlas rejects a tile when its bounding sphere lies below the ellipsoid horizon. Remaining spheres are tested against the camera view cone.

For geometric error g, distance d, viewport height H, and vertical field of view θ, screen-space error is

$$\mathrm{SSE} = \frac{gH}{2d\tan(\theta/2)}.$$

A priority queue refines the highest-utility node while respecting a hard tile-count bound. Route-corridor nodes use a lower effective SSE threshold so limited refinement capacity reaches future navigation content before unrelated detail. The selector records visited nodes, horizon and view culls, refined nodes, route-biased nodes, and deepest selected level.

In a 640×360 MoltenVK smoke run on an Apple M2 Pro, the bounded selector chose 382 leaves, visited 562 nodes, horizon-culled 41 nodes, reached level 4, and marked a route-biased node. These are execution diagnostics, not a global quality result.

5 Streaming and Integrity

The runtime accepts file, memory, HTTP, and SQLite regional-pack sources through one cancellable asynchronous interface. HTTP supports ETags, conditional requests, timeout, redirect limits, request cancellation, and byte-range content through the gateway. The disk cache performs atomic temporary writes, quota-aware least-recently-used eviction, pinning, and metadata updates.

Every payload is hashed from its actual bytes using an internal SHA-256 implementation. Expected, transport, cache, and pack checksums are compared against the calculated digest rather than merely against each other. A corrupt cache entry becomes a miss; a corrupt pack row fails explicitly. No unavailable tile or checksum error is silently accepted.

The '.vxa' format is a transactional SQLite archive. Its primary tile key contains layer, face, level, x, and y; rows store payload, ETag, and SHA-256. Additional tables store metadata, offline points of interest, and named routing/transit assets. Writers use a transaction, checkpoint WAL state, and return to a portable delete journal before closing. Readers use independent read-only connections.

The checked Connaught Place compatibility pack contains 453 GLB rows: 151 each at levels 18, 19, and 20. The five reference manifests cover Delhi NCR, Greater London, Tokyo metro, New York metro, and the Swiss Alps. They contain ten layer contracts and six implicit cube roots. CI regenerates each manifest byte-for-byte.

6 Navigation Contract

Four C++ interfaces isolate consumer code from providers: `SearchProvider`, `RouteProvider`, `TransitProvider`, and `TrafficProvider`. The normalized gateway exposes `/v1/search`, `/v1/reverse`, `/v1/route`, `/v1/transit`, `/v1/traffic`, `/v1/status`, and byte-range content. `GatewayNavigationProvider` consumes this contract through libcurl.

A deterministic replay provider supplies place, road, transit, and traffic fixtures. The rendered globe requests the replay route and constructs a tangent-plane ribbon above the ellipsoid. Segments are geodesically subdivided before triangulation, so long routes follow the surface instead of cutting through the globe.

The gateway never logs API keys. Its replay mode is a complete offline contract fixture. Self-hosted deployments can place Pelias, Valhalla, OpenTripPlanner, traffic ingestion, and object storage behind the same client-facing schema. Offline '.vxa' packs retain local POI search and can carry regional graph extracts.

7 Route-Predictive Controller

For candidate tile t, route polyline R, and corridor radius σ, the controller first computes the minimum ECEF distance $d(t, R)$ after subtracting the tile radius. Spatial route probability is

$$p_s(t) = \exp\left(-\frac{d(t,R)^2}{2\sigma^2}\right).$$

Distance along the route and expected speed estimate seconds until needed $\tau(t)$. A bounded look-ahead factor yields

$$p_r(t) = p_s(t)\left(0.35 + 0.65 \max\left(0, 1 - \frac{\tau(t)}{T}\right)\right).$$

Visual utility combines semantic weight w_s, quality gain Q, visibility confidence V, SSE, route probability, and deadline pressure:

$$U(t) = w_s Q(0.25 + V) + p_r W_{\text{route}} W_{\text{deadline}} + 0.02\,\text{SSE}.$$

Cost includes one unit of fixed scheduling work, upload MiB, measured or estimated GPU geometry cost, and lighting cost. Candidates are sorted deterministically by U/C and tile key.

Selection enforces resident memory, per-frame upload, maximum changes, and speculative-waste budgets. Frame-time pressure comes from an exponentially weighted moving average and reduces the change limit above the target. Route deviation sets route probability to zero immediately.

Seven policies expose the controller's assumptions:

- distance-only: current visibility and geometric quality;
- velocity-only: forward extrapolation from camera velocity;

- route-only: route probability without semantic weights;
- route-semantics: route and semantic weights;
- geometry-only: lighting cost removed;
- lighting-only: upload cost removed;
- full Atlas: route, semantics, geometry, upload, and lighting.

8 Renderer Integration

Platform-specific Vulkan ownership is isolated behind `VulkanSurfaceHost`: framebuffer extent, resize state, event waiting, instance extensions, and surface creation. Device and renderer code no longer imports GLFW. The GLFW implementation remains the desktop host while Qt, Android `ANativeWindow`, and headless hosts share the same contract.

Atlas is split into CMake libraries for core coordinates/data, network transport, navigation, research control, rendering primitives, and streaming. `VulkaxAtlasCore` remains a compatibility aggregate. The existing `LveEngine` executable preserves all BEACON and GeoBEACON regression modes; `VulkaxAtlas` enables the globe entrypoint.

Offscreen exact-reference readback waits on the fence for the submitted frame. It no longer calls `vkDeviceWaitIdle` inside the measured frame loop. Geo tile resources retire after more than the maximum frames in flight; shutdown waits the graphics queue. Swapchain recreation and final application shutdown retain explicit synchronization.

9 Methodology

The checked controller fixture contains 183 unique level-16 candidates around a 3.2 km Delhi route and surrounding distractors. A 4 MiB resident budget holds only 64 tiles. Upload is limited to 4 MiB/s and two changes per frame; each tile is 64 KiB. Each policy receives the same 600-frame fixed path, fixed 1/60 s timestep, seed 1337, costs, route, and candidate set.

At each frame, the benchmark checks the current and three near-future route demands before applying new decisions. A missing required key is a deadline miss. A resident tile never observed in the route-demand set contributes to wasted prefetch bytes. The output includes every frame, cumulative misses, selected and resident counts, upload and speculative bytes, semantic utility, route quality, and budget violation.

This experiment isolates scheduling logic. Its measurement class is `analytical-scheduler-simulation`; it must not be merged with Vulkan timings. Hardware experiments use the separate renderer path, GPU query pools, deterministic cameras, warm-up periods, exact diffuse captures, and raw CSV output.

10 Results

The fixture supports the preregistered existence claim: route-aware control beats distance-only control on route-corridor misses under at least one constrained workload. Route-only

Table 1: Constrained 600-frame scheduler fixture.

Policy	Deadline misses	Wasted bytes
Distance only	644	3,145,728
Velocity only	204	2,818,048
Route only	8	2,686,976
Route + semantics	4	2,686,976
Geometry only	644	3,145,728
Lighting only	644	3,145,728
Full Atlas	4	2,686,976

accounts for most of the reduction; semantic weighting resolves the remaining tie in this fixture. Velocity prediction improves over distance but cannot anticipate turns as effectively as the recorded route. Full Atlas and route-plus-semantics tie because this experiment does not execute actual lighting or geometry kernels; that negative discrimination is expected and motivates hardware coupled experiments.

The full policy also reduces wasted resident bytes by 458,752 (14.6%) relative to distance-only. The absolute values are fixture-dependent and are not presented as consumer-navigation performance. Raw rows and the JSON summary regenerate the checked SVG figures without third-party plotting packages.

11 Verification

The current automated suite covers BEACON bounds and scan behavior, WGS84/ECEF round trips, cube addressing, manifest validation, SHA-256 vectors, cache and pack I/O, offline POI search, route scheduling, runtime completion, replay and gateway providers, route mesh construction, bounded globe selection, deterministic OSM tools, HTTP endpoint/range/ETag contracts, five reference manifest regenerations, and the route-controller stress relation.

Linux CI builds release code, compiles shaders, runs CTest, builds the gateway container, resolves Lavapipe explicitly, performs GeoBEACON and Atlas Vulkan smoke runs under Xvfb, and uploads raw measurements. Local validation covers MoltenVK timestamp support and Vulkan CPU-fallback classification on the second ICD.

12 Limitations and Scope

The checked five-region artifacts are manifests and deterministic source contracts rather than committed full regional terrain databases; generated regional and planet payloads are intentionally outside Git. Live data quality depends on configured OSM, terrain, GTFS, and traffic sources. Traffic is optional BYOK and deterministic replay remains available.

The controller result reported here is analytical. Consumer usability, transit completeness, cross-platform shell parity, and planet-scale service load require separate user, hardware, and deployment studies. Formal BEACON error bounds exclude shadows, transparency, and strict specular guarantees. Proprietary imagery, photogrammetry, and neural rendering are outside the artifact's open procedural-data scope.

13 Conclusion

Vulkax Atlas turns the checked GeoBEACON renderer into a planet-coordinate, streamable, navigable research substrate. Its central contribution is a transparent route-predictive controller that jointly values future geometry, semantics, upload, memory, and lighting. The constrained fixture demonstrates why route knowledge matters and identifies which components produce the effect. Checksummed data boundaries, deterministic manifests, raw rows, explicit evidence classes, and preserved negative cases keep the result reproducible and falsifiable.

14 Artifact Reproduction Checklist

The repository is designed to make each evidence class independently reproducible. A clean release build first compiles the C++20 component libraries, the Vulkan application, `atlas-build`, `atlas-benchmark`, and every GLSL stage. CTest then executes core BEACON checks, Atlas coordinate and scheduling checks, gateway contracts, manifest regeneration, GeoBEACON preprocessing checks, and the constrained controller relation. The test runner fails if full Atlas does not beat distance-only control in the preregistered constrained fixture.

Dataset reproduction begins from a region source manifest rather than from unrecorded local state. For each of the five reference regions, `atlas-build generate-manifest` writes dataset version 2 and a six-root implicit 3D Tiles descriptor. CI compares the generated bytes with the checked files. Connaught Place pack reproduction starts from the committed OSM-derived GeoBEACON manifest and GLBs, writes a transactional '.vxa' archive, checks SQLite integrity, and verifies 151 rows at each of three levels. Full generated regional and planet payloads remain external artifacts because committing them would make normal source control unusable.

Research reproduction runs `atlas-benchmark` with frame count 600 and seed 1337. It writes `frames.csv` before aggregation and records the evidence class in every row. The figure script consumes only `summary.json` and emits the checked deadline-miss and wasted-prefetch SVGs. No plotting library, network request, or GPU is required for this scheduler experiment. Vulkan hardware measurements use the separate `VulkaxAtlas` or `LveEngine` entrypoint and preserve device, driver, shader hash, timing source, resolution, warm-up count, and camera path in the run manifest.

For visual verification, the Atlas entrypoint loads the deterministic road route, constructs the ellipsoid-following ribbon, and displays permanent OSM attribution. The measured GeoBEACON path can add exact and tested offscreen targets to the same submitted command buffer. Readback waits on that frame's fence, writes reference/test PPM files, and reports actual image metrics separately from the conservative diffuse bound. The final PDF is compiled from this checked LaTeX source; page count and rendered pages are inspected after compilation.

References

[1] Open Geospatial Consortium, "3D Tiles Specification 1.1," 2022. https://docs.ogc.org/cs/22-025r4/22-025r4.html

[2] OpenStreetMap Foundation, "Copyright and License." https://www.openstreetmap.org/copyright

[3] OpenStreetMap Wiki, "Planet.osm diffs." https://wiki.openstreetmap.org/wiki/Planet.osm/diffs

[4] Pelias, "Open-source geocoder documentation." https://github.com/pelias/documentation

[5] Valhalla, "Open-source routing engine." https://valhalla.github.io/valhalla/

[6] OpenTripPlanner, "Open-source multimodal trip planner." https://docs.opentripplanner.org/

[7] Boksansky et al., "Rendering Many Lights with Grid-Based Reservoirs," 2021. https://research.nvidia.com/labs/rtr/publication/boksansky2021rendering/